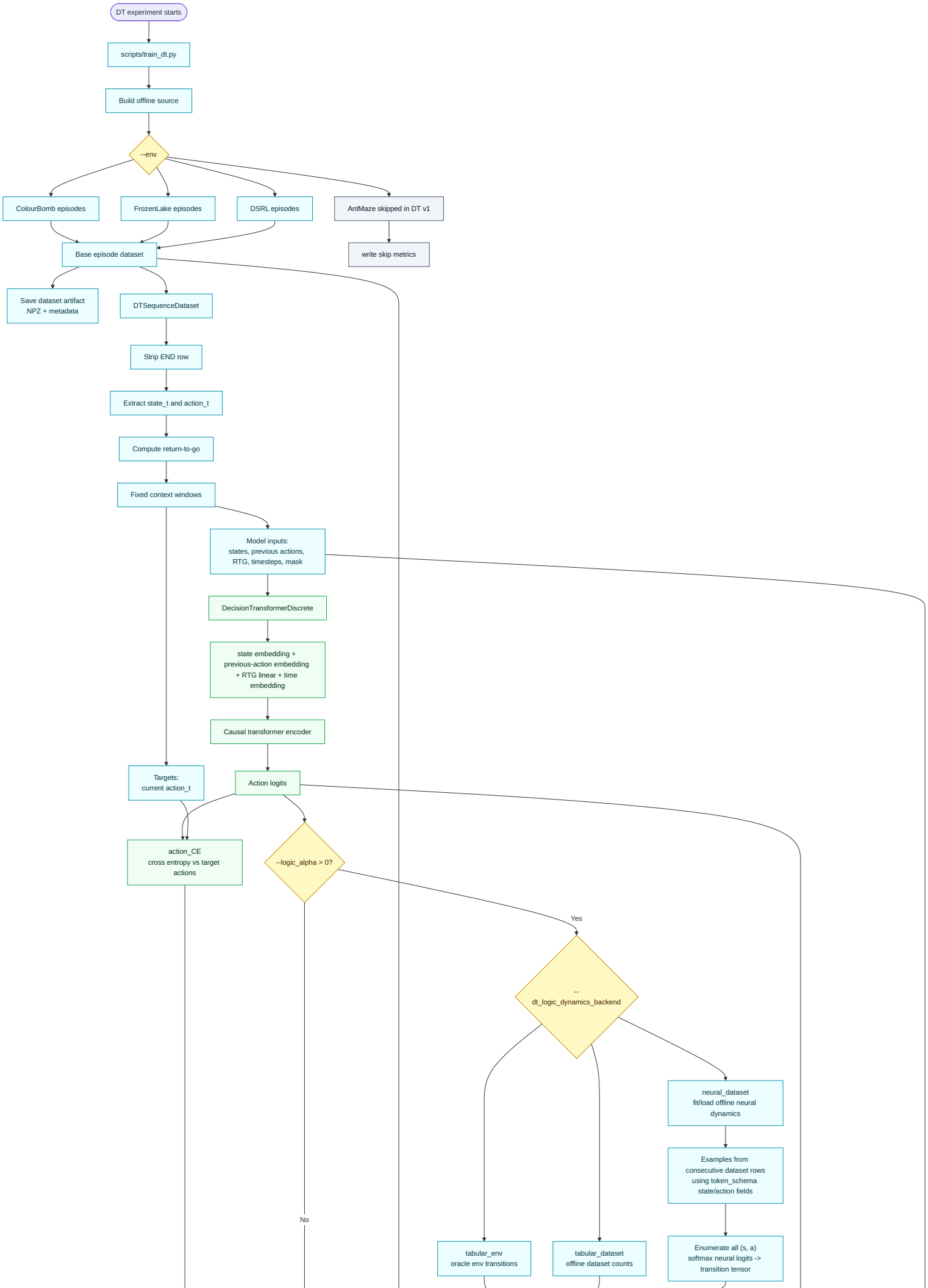
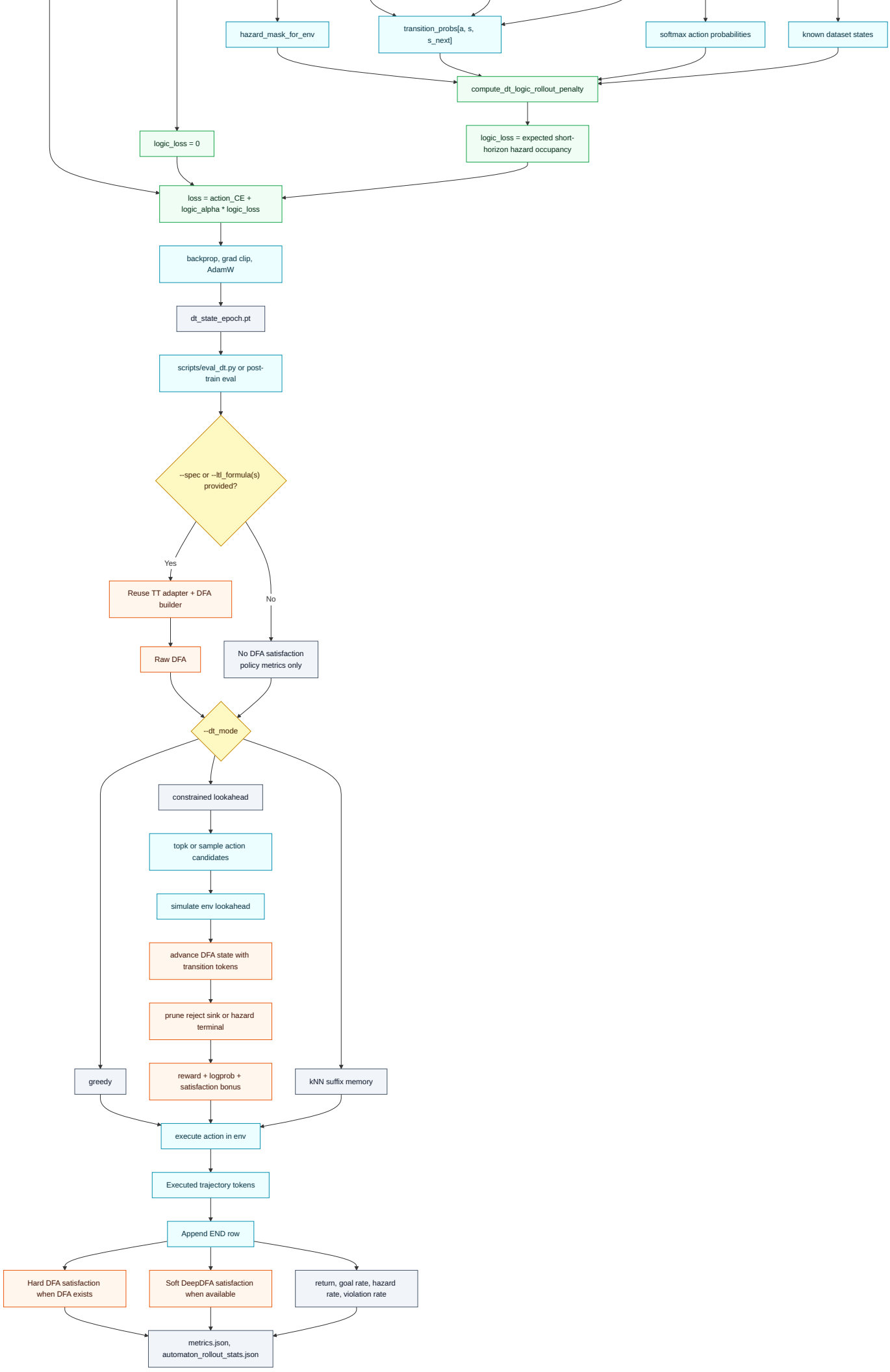

DT experiment starts
scripts/train_dt.py
Build offline source
--env
ColourBomb episodes
FrozenLake episodes
DSRL episodes
AntMaze skipped in DT v1
write skip metrics
Base episode dataset
Save dataset artifact NPZ + metadata
DTSequenceDataset
Strip END row
Extract state_t and action_t
Compute return-to-go
Fixed context windows
Model inputs: states, previous actions, RTG, timesteps, mask
DecisionTransformerDiscrete
state embedding + previous-action embedding + RTG linear + time embedding
Causal transformer encoder
Targets: current action_t
Action logits
action_CE cross entropy vs target actions
--logic_alpha > 0?
Yes
No
-- dt_logic_dynamics_backend
neural_dataset fit/load offline neural dynamics
Examples from consecutive dataset rows using token_schema state/action fields
Enumerate all (s, a) softmax neural logits -> transition tensor
tabular_env oracle env transitions
tabular_dataset offline dataset counts


hazard_mask_for_env
transition_probs[a, s, s_next]
softmax action probabilities
known dataset states
compute_dt_logic_rollout_penalty
logic_loss = 0
logic_loss = expected short-horizon hazard occupancy
loss = action_CE + logic_alpha * logic_loss
backprop, grad clip, AdamW
dt_state_epoch.pt
scripts/eval_dt.py or post-train eval
--spec or --ltl_formula(s) provided?
Yes
No
Reuse TT adapter + DFA builder
Raw DFA
No DFA satisfaction policy metrics only
--dt_mode
constrained lookahead
topk or sample action candidates
simulate env lookahead
advance DFA state with transition tokens
prune reject sink or hazard terminal
greedy
reward + logprob + satisfaction bonus
kNN suffix memory
execute action in env
Executed trajectory tokens
Append END row
Hard DFA satisfaction when DFA exists
Soft DeepDFA satisfaction when available
return, goal rate, hazard rate, violation rate
metrics.json, automaton_rollout_stats.json